



# Bug Report — Skill 05 Markdown Parser

*Nexostrat — Solo para uso de Ricardo · 26 May 2026*

## Tolerancia insuficiente a líneas en blanco entre ###

### TABLA\_OPORTUNIDADES y la tabla

---

**Severidad:** Media. Falla silenciosa que produce reporte cliente-visible incompleto sin warning ni error.

**Reproducible:** Sí, determinístico.

**Descubierto:** 2026-05-26 generando reporte para Trixx Logistics.

**Impacto:** El reporte se genera "exitosamente" pero sin inventario de oportunidades, sin las dos gráficas 5×5 y sin la matriz 2×2. Si nadie revisa el .docx antes de enviarlo, el cliente recibe un reporte mutilado.

## 1. Síntoma observado

---

Al correr el script con el markdown de Trixx, la salida fue:

El reporte se generó (exit code 0, no errores), pero con **0 oportunidades detectadas**. El .docx resultante no incluyó:

- La tabla del inventario de oportunidades (renderizado vacío).
- La Gráfica 1 (Impacto vs. Complejidad — 5×5).
- La Gráfica 2 (Impacto vs. Riesgo — 5×5).
- La matriz de priorización 2×2 de la sección 4.
- Los dos Quick Wins seleccionados automáticamente por score.

El reporte se entregó sin la mitad de su contenido analítico crítico.

## 2. Diagnóstico — flujo del parser

---

El script `skills/05_opportunity_report/scripts/generate_docx.py` busca el marcador `###` `TABLA_OPORTUNIDADES` para activar el modo de lectura de tabla de oportunidades. La lógica relevante (líneas 757-770 + 808-812):

**Comportamiento:** una vez que `in_opp_table` está activo, la PRIMERA línea que no empiece con `|` se interpreta como "la tabla terminó". El parser llama entonces a `parse_opportunity_table()` con `opp_table_lines` aún vacío.

## 3. Causa raíz

---

El markdown que generé seguía la convención estándar CommonMark de separar el encabezado de su contenido con una línea en blanco:

El parser leyó:

1. Línea ### TABLA\_OPORTUNIDADES → activó `in_opp_table = True`, `opp_table_lines = []`.
2. Siguiendo línea: **línea en blanco** → no empieza con `|` → entró al `else` → llamó `parse_opportunity_table([])` → devolvió `[]`.
3. Marcó `in_opp_table = False`.
4. Las líneas reales de la tabla (que venían después de la línea en blanco) fueron interpretadas como una tabla genérica normal — se renderizaron como tabla simple pero sin los puntajes parseados, sin gráficas, sin matriz.

**Causa raíz formal:** el parser asume que el contenido de un bloque ###

TABLA\_OPORTUNIDADES empieza inmediatamente en la línea siguiente al encabezado, sin tolerar líneas en blanco intermedias.

## 4. Fix aplicado (workaround)

Editó el markdown para pegar la tabla inmediatamente después del encabezado, sin línea en blanco:

Re-ejecutando el script con el markdown corregido, la salida fue:

El reporte se completó con las 10 oportunidades, las dos gráficas 5×5, la matriz 2×2 y los dos Quick Wins identificados automáticamente. PDF final: 287 KB versus 219 KB de la primera corrida (la diferencia son los gráficos faltantes).

## 5. Fix permanente recomendado

El parser debe saltar líneas en blanco mientras `in_opp_table` está activo y aún no se ha visto la primera línea de tabla. Cambio sugerido en `process_markdown()` alrededor de la línea 757:

**Justificación:** la condición `not opp_table_lines` asegura que solo se saltan líneas en blanco al inicio (antes de que se haya acumulado contenido de tabla). Una línea en blanco después de comenzar la tabla sí debe terminarla. Esto preserva la semántica actual y solo agrega tolerancia al inicio.

**Cambio adicional sugerido — warning explícito cuando se llama `parse_opportunity_table` con lista vacía:**

Esto convierte el modo de falla silencioso en un warning visible. El reporte aún se genera (no breaking change), pero el operador ve la alerta y puede investigar antes de entregarlo al cliente.

## 6. Análisis de impacto

| Dimensión | Estado actual | Con fix |
|-----------|---------------|---------|
|-----------|---------------|---------|

|                                       |                                                         |                                               |
|---------------------------------------|---------------------------------------------------------|-----------------------------------------------|
| Robustez de markdown                  | Frágil — convención CommonMark estándar rompe el parser | Tolerante — markdown estándar funciona        |
| Modo de falla                         | Silencioso — exit 0, reporte vacío                      | Visible — warning si la tabla no parseó nada  |
| Riesgo de entrega errónea a cliente   | Alto si nadie revisa el .docx final                     | Bajo                                          |
| Esfuerzo de implementación            | —                                                       | ~5 líneas de código + 1 unit test             |
| Compatibilidad con markdown existente | —                                                       | Backwards compatible (solo agrega tolerancia) |

## 7. Recomendaciones operativas

5. **Aplicar el patch del § 5** en `skills/05_opportunity_report/scripts/generate_docx.py`. Tiempo estimado: 15 minutos incluyendo test manual.
6. **Agregar un fixture de regresión** en `skills/05_opportunity_report/tests/` con un markdown que tenga línea en blanco entre `### TABLA_OPORTUNIDADES` y la tabla, verificando que el parseo extrae las filas correctamente.
7. **Auditar los otros parsers similares** en los scripts `generate_docx.py` de Skills 01-04. Si alguno tiene el mismo patrón estricto (heading marcador seguido inmediatamente de contenido), aplicar la misma tolerancia.
8. **Considerar agregar un warning genérico** cuando cualquier sección esperada del template del skill (`TABLA_OPORTUNIDADES`, callouts, etc.) no genere salida — falla silenciosa de cualquier marcador es el modo de error más peligroso para entregables cliente-visibles.

## 8. Conclusión

Bug menor en términos de complejidad técnica (~5 líneas de fix), pero con impacto potencial alto: produce un reporte cliente-visible incompleto sin ninguna señal visible al operador. La probabilidad de recurrencia es alta porque la convención de línea en blanco entre encabezado y contenido es estándar en markdown.

El fix es backwards-compatible y agrega defensas explícitas (warning) para futuros casos donde un marcador del template no produzca salida.

*Reportado por Client-Owner durante la generación del reporte de Trixx Logistics 2026-05-26.  
Memo dirigido a Skills-Master per protocolo Cross-Folder Memo (ADR-013).*